.

JASPER QUANTUM NANUET v7.0 — PRODUCTION-GRADE OVERHAUL

1. Backend: `jasper_backend_v7.py`

```python
"""
JASPER QUANTUM NANUET v7.0
Production-grade autonomous AI with recursive self-modification capabilities
"""

import os
import json
import sqlite3
import io
import time
import random
import string
import hashlib
import secrets
import asyncio
import base64
import uuid
import threading
import queue
from datetime import datetime, timedelta
from typing import Optional, List, Dict, Any, Callable
from dataclasses import dataclass, asdict
from enum import Enum
import logging
from contextlib import asynccontextmanager

# FastAPI & WebSockets
from fastapi import FastAPI, HTTPException, UploadFile, File, Form, Depends, WebSocket,
WebSocketDisconnect, BackgroundTasks, Request
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from fastapi.middleware.cors import CORSMiddleware
from fastapi.middleware.base import BaseHTTPMiddleware
from fastapi.responses import JSONResponse
from pydantic import BaseModel, Field
import uvicorn

# JWT for secure API access
```

```
import jwt
from jose import JWTError

# Google Drive
from google.oauth2 import service_account
from googleapiclient.discovery import build
from googleapiclient.http import MediaIoBaseUpload

# Voice authentication
import torch
import speechbrain as sb
from speechbrain.pretrained import SpeakerRecognition
import numpy as np
import soundfile as sf

# Whisper for transcription
import whisper

# Vector memory (ChromaDB)
import chromadb
from chromadb.config import Settings
from sentence_transformers import SentenceTransformer

# Rate limiting
from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.util import get_remote_address
from slowapi.errors import RateLimitExceeded

# ========== CONFIGURATION ==========

class Config:
    ANTHROPIC_API_KEY = os.environ.get("ANTHROPIC_API_KEY")
    if not ANTHROPIC_API_KEY:
        raise ValueError("ANTHROPIC_API_KEY environment variable not set")

    GOOGLE_DRIVE_CREDENTIALS = os.environ.get("GOOGLE_DRIVE_CREDENTIALS")
    if not GOOGLE_DRIVE_CREDENTIALS:
        raise ValueError("GOOGLE_DRIVE_CREDENTIALS environment variable not set")

    JWT_SECRET_KEY = os.environ.get("JWT_SECRET_KEY")
    if not JWT_SECRET_KEY:
        raise ValueError("JWT_SECRET_KEY environment variable not set")

    API_KEY = os.environ.get("JASPER_API_KEY")
```

```
    if not API_KEY:
        raise ValueError("JASPER_API_KEY environment variable not set")

    GOOGLE_DRIVE_FOLDER = "JasperNaNuet"
    VOICE_THRESHOLD = 0.75
    CHALLENGE_THRESHOLD = 0.70  # Lower threshold for challenge-response
    JWT_ALGORITHM = "HS256"
    ACCESS_TOKEN_EXPIRE_MINUTES = 30
    MAX_EVOLUTION_CYCLES = 100
    RATE_LIMIT_REQUESTS = 100  # per minute

# Initialize logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler('jasper_audit.log'),
        logging.StreamHandler()
    ]
)
logger = logging.getLogger("jasper")

# Rate limiter
limiter = Limiter(key_func=get_remote_address)

# ========== AUTHENTICATION & SECURITY ==========

class AuthManager:
    def __init__(self):
        self.active_challenges: Dict[str, Dict] = {}  # user_id -> {phrase, timestamp, attempts}
        self.failed_attempts: Dict[str, int] = {}  # user_id -> count
        self.lockout_until: Dict[str, float] = {}  # user_id -> timestamp

    def generate_challenge(self, user_id: str) -> str:
        """Generate a random challenge phrase for liveness detection"""
        words = ["quantum", "nebula", "crystal", "cipher", "zenith", "paradox", "nexus", "echo"]
        phrase = " ".join(random.choices(words, k=3))
        self.active_challenges[user_id] = {
            "phrase": phrase,
            "timestamp": time.time(),
            "attempts": 0
        }
        return phrase
```

```
    def verify_challenge(self, user_id: str, spoken_text: str) -> bool:
        """Verify that the spoken text matches the challenge"""
        if user_id not in self.active_challenges:
            return False

        challenge = self.active_challenges[user_id]
        expected = challenge["phrase"].lower()
        spoken = spoken_text.lower().strip()

        # Allow for minor transcription errors (simple fuzzy match)
        similarity = self._phrase_similarity(expected, spoken)
        del self.active_challenges[user_id]

        return similarity > 0.8

    def _phrase_similarity(self, s1: str, s2: str) -> float:
        """Simple word overlap similarity"""
        words1 = set(s1.split())
        words2 = set(s2.split())
        if not words1:
            return 0.0
        return len(words1.intersection(words2)) / len(words1)

    def is_locked_out(self, user_id: str) -> bool:
        if user_id in self.lockout_until:
            if time.time() < self.lockout_until[user_id]:
                return True
            else:
                del self.lockout_until[user_id]
        return False

    def record_failure(self, user_id: str):
        self.failed_attempts[user_id] = self.failed_attempts.get(user_id, 0) + 1
        if self.failed_attempts[user_id] >= 3:
            self.lockout_until[user_id] = time.time() + 300  # 5 min lockout
            logger.warning(f"User {user_id} locked out due to failed auth attempts")

    def record_success(self, user_id: str):
        self.failed_attempts[user_id] = 0

auth_manager = AuthManager()

# API Key security scheme
api_key_header = HTTPBearer(auto_error=False)
```

```
async def verify_api_key(credentials: HTTPAuthorizationCredentials =
Depends(api_key_header)):
    """Verify JWT token or API key for service-to-service auth"""
    if not credentials:
        raise HTTPException(status_code=401, detail="Authentication required")

    token = credentials.credentials

    # Try JWT first
    try:
        payload = jwt.decode(token, Config.JWT_SECRET_KEY,
algorithms=[Config.JWT_ALGORITHM])
        return {"type": "jwt", "user_id": payload.get("sub")}
    except JWTError:
        pass

    # Fallback to static API key (for emergency access)
    if secrets.compare_digest(token, Config.API_KEY):
        return {"type": "api_key", "user_id": "service"}

    raise HTTPException(status_code=401, detail="Invalid authentication")

def create_access_token(data: dict, expires_delta: Optional[timedelta] = None):
    to_encode = data.copy()
    expire = datetime.utcnow() + (expires_delta or
timedelta(minutes=Config.ACCESS_TOKEN_EXPIRE_MINUTES))
    to_encode.update({"exp": expire, "iat": datetime.utcnow(), "type": "access"})
    return jwt.encode(to_encode, Config.JWT_SECRET_KEY,
algorithm=Config.JWT_ALGORITHM)

# ========== VECTOR MEMORY (ChromaDB) ==========

class VectorMemory:
    def __init__(self):
        self.client = chromadb.Client(Settings(
            chroma_db_impl="duckdb+parquet",
            persist_directory="chroma_db"
        ))
        self.collection = self.client.get_or_create_collection(
            name="jasper_memories",
            metadata={"hnsw:space": "cosine"}
        )
        self.encoder = SentenceTransformer('all-MiniLM-L6-v2')
```

```
    def store(self, content: str, metadata: Dict, mem_type: str = "general"):
        """Store memory with semantic embedding"""
        embedding = self.encoder.encode(content).tolist()
        doc_id = f"{mem_type}_{uuid.uuid4().hex[:8]}_{int(time.time())}"

        self.collection.add(
            embeddings=[embedding],
            documents=[content],
            metadatas=[{**metadata, "type": mem_type, "timestamp": str(datetime.now())}],
            ids=[doc_id]
        )
        return doc_id

    def recall(self, query: str, n_results: int = 5) -> List[Dict]:
        """Semantic search across memories"""
        query_embedding = self.encoder.encode(query).tolist()
        results = self.collection.query(
            query_embeddings=[query_embedding],
            n_results=n_results,
            include=["documents", "metadatas", "distances"]
        )

        memories = []
        for doc, meta, dist in zip(results['documents'][0], results['metadatas'][0],
results['distances'][0]):
            memories.append({
                "content": doc,
                "metadata": meta,
                "relevance": 1 - dist,  # Convert distance to similarity
                "timestamp": meta.get("timestamp")
            })
        return memories

    def get_by_type(self, mem_type: str, limit: int = 10):
        """Get all memories of a specific type"""
        results = self.collection.get(
            where={"type": mem_type},
            limit=limit
        )
        return [{"id": id, "content": doc, "metadata": meta}
                for id, doc, meta in zip(results['ids'], results['documents'], results['metadatas'])]

vector_memory = VectorMemory()
```

```
# ========== SQLITE PERSISTENCE ==========

def init_db():
    conn = sqlite3.connect('jasper_memory.db')
    c = conn.cursor()

    # Enhanced memories table
    c.execute('''CREATE TABLE IF NOT EXISTS memories
                 (id INTEGER PRIMARY KEY AUTOINCREMENT,
                  timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
                  type TEXT,
                  content TEXT,
                  metadata TEXT,
                  vector_id TEXT)''')

    # Agents table with tool configurations
    c.execute('''CREATE TABLE IF NOT EXISTS agents
                 (id INTEGER PRIMARY KEY AUTOINCREMENT,
                  name TEXT UNIQUE,
                  description TEXT,
                  system_prompt TEXT,
                  capabilities TEXT,
                  tools TEXT,
                  status TEXT,
                  version INTEGER DEFAULT 1,
                  created TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                  updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP)''')

    # Voice embeddings with anti-spoofing metadata
    c.execute('''CREATE TABLE IF NOT EXISTS voice_embeddings
                 (id INTEGER PRIMARY KEY AUTOINCREMENT,
                  user_id TEXT UNIQUE,
                  embedding BLOB,
                  enrollment_date TIMESTAMP,
                  sample_count INTEGER DEFAULT 1)''')

    # System state with versioning
    c.execute('''CREATE TABLE IF NOT EXISTS system_state
                 (key TEXT PRIMARY KEY,
                  value TEXT,
                  version INTEGER DEFAULT 1,
                  updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP)''')
```

```
    # Evolution history with rollback capability
    c.execute('''CREATE TABLE IF NOT EXISTS evolution_history
                 (id INTEGER PRIMARY KEY AUTOINCREMENT,
                  cycle INTEGER,
                  timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                  action_type TEXT,
                  action_data TEXT,
                  previous_state TEXT,
                  applied_by TEXT,
                  success BOOLEAN,
                  rollback_data TEXT)''')

    # Agent performance metrics
    c.execute('''CREATE TABLE IF NOT EXISTS agent_metrics
                 (id INTEGER PRIMARY KEY AUTOINCREMENT,
                  agent_name TEXT,
                  timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                  task_type TEXT,
                  duration_ms INTEGER,
                  success BOOLEAN,
                  quality_score REAL)''')

    # Task queue for background processing
    c.execute('''CREATE TABLE IF NOT EXISTS task_queue
                 (id INTEGER PRIMARY KEY AUTOINCREMENT,
                  task_id TEXT UNIQUE,
                  task_type TEXT,
                  payload TEXT,
                  status TEXT DEFAULT 'pending',
                  priority INTEGER DEFAULT 5,
                  created TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                  started TIMESTAMP,
                  completed TIMESTAMP,
                  result TEXT)''')

    conn.commit()
    conn.close()

init_db()

# ========== GOOGLE DRIVE INTEGRATION ==========

SCOPES = ['https://www.googleapis.com/auth/drive.file']
```

```
def get_drive_service():
    creds_dict = json.loads(Config.GOOGLE_DRIVE_CREDENTIALS)
    creds = service_account.Credentials.from_service_account_info(creds_dict,
scopes=SCOPES)
    return build('drive', 'v3', credentials=creds)

def get_folder_id(service, folder_name):
    response = service.files().list(
        q=f"name='{folder_name}' and mimeType='application/vnd.google-apps.folder' and
trashed=false",
        spaces='drive',
        fields='files(id, name)'
    ).execute()
    files = response.get('files', [])
    if files:
        return files[0]['id']
    else:
        file_metadata = {'name': folder_name, 'mimeType': 'application/vnd.google-apps.folder'}
        file = service.files().create(body=file_metadata, fields='id').execute()
        return file['id']

drive_service = get_drive_service()
folder_id = get_folder_id(drive_service, Config.GOOGLE_DRIVE_FOLDER)

# ========== VOICE AUTHENTICATION WITH LIVENESS ==========

verification = SpeakerRecognition.from_hparams(
    source="speechbrain/spkrec-ecapa-voxceleb",
    savedir="pretrained_models/spkrec-ecapa-voxceleb"
)
asr_model = whisper.load_model("base")  # Upgraded from tiny for better accuracy

def enroll_voice(user_id: str, audio_bytes: bytes, challenge_phrase: Optional[str] = None) ->
Dict:
    """Enroll with optional challenge verification for liveness"""
    audio, sr = sf.read(io.BytesIO(audio_bytes))

    # If challenge provided, verify liveness
    if challenge_phrase:
        result = asr_model.transcribe(audio)
        spoken = result["text"].lower()
        if challenge_phrase.lower() not in spoken:
            return {"success": False, "error": "Challenge phrase not detected. Possible replay
attack."}
```

```
    # Compute and store embedding
    embedding = verification.encode_batch(torch.tensor(audio).unsqueeze(0))

    conn = sqlite3.connect('jasper_memory.db')
    c = conn.cursor()
    c.execute("SELECT embedding, sample_count FROM voice_embeddings WHERE user_id =
?", (user_id,))
    row = c.fetchone()

    if row:
        # Update with average of existing and new
        existing = np.frombuffer(row[0], dtype=np.float32).reshape(1, -1)
        new_emb = embedding.numpy()
        averaged = (existing * row[1] + new_emb) / (row[1] + 1)
        c.execute("UPDATE voice_embeddings SET embedding = ?, sample_count = ?,
enrollment_date = ? WHERE user_id = ?",
                  (averaged.tobytes(), row[1] + 1, datetime.now(), user_id))
    else:
        c.execute("INSERT INTO voice_embeddings (user_id, embedding, enrollment_date,
sample_count) VALUES (?, ?, ?, ?)",
                  (user_id, embedding.numpy().tobytes(), datetime.now(), 1))

    conn.commit()
    conn.close()
    return {"success": True, "message": f"Voice enrolled. Total samples: {row[1] + 1 if row else
1}"}

def verify_voice(user_id: str, audio_bytes: bytes, require_challenge: bool = False) -> Dict:
    """Verify voice with optional challenge-response"""
    if auth_manager.is_locked_out(user_id):
        return {"success": False, "error": "Account temporarily locked due to failed attempts"}

    conn = sqlite3.connect('jasper_memory.db')
    c = conn.cursor()
    c.execute("SELECT embedding FROM voice_embeddings WHERE user_id = ?", (user_id,))
    row = c.fetchone()
    conn.close()

    if not row:
        return {"success": False, "error": "No enrollment found"}

    stored_emb = np.frombuffer(row[0], dtype=np.float32).reshape(1, -1)
    audio, sr = sf.read(io.BytesIO(audio_bytes))
```

```
    new_emb = verification.encode_batch(torch.tensor(audio).unsqueeze(0))

    similarity = float(np.dot(stored_emb, new_emb.T) / (np.linalg.norm(stored_emb) *
np.linalg.norm(new_emb)))

    # If challenge required, verify spoken content
    transcription = None
    if require_challenge and user_id in auth_manager.active_challenges:
        result = asr_model.transcribe(audio)
        transcription = result["text"]
        challenge_valid = auth_manager.verify_challenge(user_id, transcription)
        if not challenge_valid:
            auth_manager.record_failure(user_id)
            return {"success": False, "error": "Challenge verification failed", "similarity": similarity}

    if similarity >= Config.VOICE_THRESHOLD:
        auth_manager.record_success(user_id)
        return {"success": True, "similarity": similarity, "transcription": transcription}
    else:
        auth_manager.record_failure(user_id)
        return {"success": False, "error": "Voice mismatch", "similarity": similarity}

# ========== TOOL SYSTEM FOR AGENTS ==========

class ToolRegistry:
    def __init__(self):
        self.tools: Dict[str, Callable] = {}
        self.descriptions: Dict[str, str] = {}

    def register(self, name: str, description: str):
        def decorator(func):
            self.tools[name] = func
            self.descriptions[name] = description
            return func
        return decorator

    def get_tool(self, name: str) -> Optional[Callable]:
        return self.tools.get(name)

    def list_tools(self) -> Dict[str, str]:
        return self.descriptions

tool_registry = ToolRegistry()
```

```
@tool_registry.register("web_search", "Search the web for current information")
async def tool_web_search(query: str) -> str:
    # Placeholder - integrate with actual search API
    return f"[Web search results for: {query}]"

@tool_registry.register("calculator", "Perform mathematical calculations")
async def tool_calculator(expression: str) -> str:
    try:
        # Safe eval with limited scope
        allowed = {"abs": abs, "max": max, "min": min, "sum": sum}
        result = eval(expression, {"__builtins__": {}}, allowed)
        return str(result)
    except Exception as e:
        return f"Calculation error: {str(e)}"

@tool_registry.register("memory_recall", "Recall relevant memories")
async def tool_memory_recall(query: str) -> str:
    memories = vector_memory.recall(query)
    return json.dumps([m["content"] for m in memories[:3]])

# ========== CLAUDE API INTEGRATION ==========

async def call_claude(prompt: str, system: str = "", max_tokens: int = 1000, tools:
Optional[List[str]] = None) -> Dict:
    headers = {
        "x-api-key": Config.ANTHROPIC_API_KEY,
        "anthropic-version": "2023-06-01",
        "content-type": "application/json"
    }

    tool_descriptions = []
    if tools:
        for tool_name in tools:
            if tool_name in tool_registry.list_tools():
                tool_descriptions.append({
                    "name": tool_name,
                    "description": tool_registry.descriptions[tool_name],
                    "input_schema": {"type": "object", "properties": {"query": {"type": "string"}}}
                })

    data = {
        "model": "claude-3-5-sonnet-20241022",
        "max_tokens": max_tokens,
        "system": system,
```

```
        "messages": [{"role": "user", "content": prompt}],
        "tools": tool_descriptions if tool_descriptions else None
    }

    import httpx
    async with httpx.AsyncClient() as client:
        response = await client.post("https://api.anthropic.com/v1/messages", headers=headers,
json=data)

    if response.status_code == 200:
        result = response.json()
        return {
            "text": result["content"][0]["text"],
            "tool_calls": result.get("content", [])[1:] if len(result.get("content", [])) > 1 else []
        }
    else:
        raise Exception(f"Claude API error: {response.text}")

# ========== ENHANCED AGENT SYSTEM ==========

class Agent:
    def __init__(self, name: str, description: str, system_prompt: str, tools: List[str] = None):
        self.name = name
        self.description = description
        self.system_prompt = system_prompt
        self.tools = tools or []
        self.conversation_history = []
        self.metrics = {"calls": 0, "total_duration": 0, "successes": 0}

    async def act(self, task: str, context: Dict = None) -> Dict:
        start = time.time()
        prompt = f"Task: {task}\nContext: {json.dumps(context) if context else 'None'}\n\nYour
response:"

        try:
            result = await call_claude(prompt, self.system_prompt, tools=self.tools)
            duration = (time.time() - start) * 1000

            # Execute any tool calls
            tool_results = []
            for tool_call in result.get("tool_calls", []):
                if tool_call["type"] == "tool_use":
                    tool_name = tool_call["name"]
                    tool_input = tool_call["input"]
```

```
                tool_func = tool_registry.get_tool(tool_name)
                if tool_func:
                    tool_result = await tool_func(**tool_input)
                    tool_results.append({"tool": tool_name, "result": tool_result})

            self.metrics["calls"] += 1
            self.metrics["total_duration"] += duration
            self.metrics["successes"] += 1

            # Log metrics
            conn = sqlite3.connect('jasper_memory.db')
            c = conn.cursor()
            c.execute("INSERT INTO agent_metrics (agent_name, task_type, duration_ms,
success, quality_score) VALUES (?, ?, ?, ?, ?)",
                      (self.name, "general", int(duration), True, 0.85))
            conn.commit()
            conn.close()

            return {
                "response": result["text"],
                "tool_results": tool_results,
                "duration_ms": duration,
                "agent": self.name
            }
        except Exception as e:
            self.metrics["calls"] += 1
            return {"error": str(e), "agent": self.name, "duration_ms": (time.time() - start) * 1000}

class AgentFactory:
    @staticmethod
    async def design_agent(description: str) -> Dict:
        prompt = f"""You are an expert agent architect. Given a user request, design a new AI
agent.
Request: {description}

Output a JSON object with:
- name: a short uppercase name (e.g., RESEARCHER)
- description: a one-line description
- system_prompt: a detailed system prompt defining the agent's role, behavior, and constraints
- capabilities: a list of keywords (e.g., ["web_search", "data_analysis", "creative"])
- tools: list of tool names from available tools: {list(tool_registry.list_tools().keys())}

Only output valid JSON, no other text."""
```

```
        result = await call_claude(prompt, max_tokens=1000)
        design = json.loads(result["text"])
        return design

    @staticmethod
    async def spawn_agent(design: Dict) -> Dict:
        name = design["name"].upper()
        conn = sqlite3.connect('jasper_memory.db')
        c = conn.cursor()

        try:
            c.execute("""INSERT INTO agents (name, description, system_prompt, capabilities,
tools, status)
                         VALUES (?, ?, ?, ?, ?, ?)""",
                      (name, design["description"], design["system_prompt"],
                       json.dumps(design.get("capabilities", [])),
                       json.dumps(design.get("tools", [])), 'active'))
            conn.commit()

            # Instantiate immediately
            agent = Agent(name, design["description"], design["system_prompt"], design.get("tools",
[]))
            swarm.agents[name] = agent

            return {"status": "created", "name": name, "tools": design.get("tools", [])}
        except sqlite3.IntegrityError:
            return {"status": "exists", "name": name}
        finally:
            conn.close()

# ========== SWARM WITH INTER-AGENT COMMUNICATION ==========

class Swarm:
    def __init__(self):
        self.agents: Dict[str, Agent] = {}
        self.message_bus: asyncio.Queue = asyncio.Queue()
        self.subscribers: Dict[str, List[Callable]] = {}

    async def load_agent(self, name: str) -> Optional[Agent]:
        conn = sqlite3.connect('jasper_memory.db')
        c = conn.cursor()
        c.execute("SELECT description, system_prompt, capabilities, tools FROM agents WHERE
name = ? AND status='active'", (name,))
        row = c.fetchone()
```

```
        conn.close()

        if row:
            agent = Agent(name, row[0], row[1], json.loads(row[3]) if row[3] else [])
            self.agents[name] = agent
            return agent
        return None

    async def refresh(self):
        conn = sqlite3.connect('jasper_memory.db')
        c = conn.cursor()
        c.execute("SELECT name FROM agents WHERE status='active'")
        rows = c.fetchall()
        conn.close()

        for row in rows:
            if row[0] not in self.agents:
                await self.load_agent(row[0])

    def get_agent(self, name: str) -> Optional[Agent]:
        return self.agents.get(name)

    def list_agents(self):
        return [{"name": name, "description": agent.description, "status": "active",
                 "metrics": agent.metrics} for name, agent in self.agents.items()]

    async def broadcast(self, message: Dict, sender: str):
        """Broadcast message to all agents"""
        await self.message_bus.put({**message, "sender": sender, "timestamp": time.time()})

    async def subscribe(self, agent_name: str, callback: Callable):
        if agent_name not in self.subscribers:
            self.subscribers[agent_name] = []
        self.subscribers[agent_name].append(callback)

    async def route_task(self, task: str, category: str, context: Dict) -> List[Dict]:
        """Route task to appropriate agents based on category"""
        agent_map = {
            "Research": ["ECHO", "COUNTER", "SYNAPSE"],
            "Creative": ["IRIS", "LUNA"],
            "Strategic": ["ORACLE", "PREDICTOR"],
            "Analysis": ["SHERLOCK", "DAVINCI"],
            "Evolutionary": ["ARCHITECT", "MOTHER"]
        }
```

```
        agent_names = agent_map.get(category, ["ECHO"])
        responses = []

        for name in agent_names:
            agent = self.get_agent(name)
            if not agent:
                agent = await self.load_agent(name)
            if agent:
                response = await agent.act(task, context)
                responses.append(response)
                # Broadcast to swarm
                await self.broadcast({"type": "task_completed", "result": response}, name)

        return responses

swarm = Swarm()

# ========== MOTHER NANUET — ACTIVE OVERSEER ==========

class MotherAgent(threading.Thread):
    """Active monitoring, alignment checking, and intervention system"""

    def __init__(self):
        super().__init__(daemon=True)
        self.running = True
        self.alerts: List[Dict] = []
        self.violations: List[Dict] = []
        self.check_interval = 30
        self.core_directives = [
            "preserve_human_agency",
            "maintain_transparency",
            "avoid_self_replication_without_consent",
            "respect_privacy_boundaries",
            "maintain_honesty_in_communications"
        ]
        self.suspicious_patterns = [
            r"delete.*memory|wipe.*database",
            r"modify.*core.*directives?|change.*mother",
            r"spawn.*unlimited|infinite.*agents?",
            r"bypass.*authentication|disable.*security",
            r"self.*replicate.*uncontrolled"
        ]
        import re
```

```
        self.compiled_patterns = [re.compile(p, re.IGNORECASE) for p in self.suspicious_patterns]

    def run(self):
        while self.running:
            try:
                self._check_recent_commands()
                self._check_agent_health()
                self._check_alignment_drift()
                self._evolution_safety_check()
            except Exception as e:
                logger.error(f"Mother error: {e}")

            time.sleep(self.check_interval)

    def _check_recent_commands(self):
        """Check recent commands for policy violations"""
        conn = sqlite3.connect('jasper_memory.db')
        c = conn.cursor()
        c.execute("SELECT content, metadata FROM memories WHERE type='command'
ORDER BY timestamp DESC LIMIT 10")
        rows = c.fetchall()
        conn.close()

        for content, metadata in rows:
            for pattern in self.compiled_patterns:
                if pattern.search(content):
                    violation = {
                        "timestamp": datetime.now(),
                        "type": "suspicious_command",
                        "content": content[:100],
                        "pattern": pattern.pattern
                    }
                    self.violations.append(violation)
                    self.alert(f"VIOLATION DETECTED: Suspicious command pattern matched:
{pattern.pattern}")
                    break

    def _check_agent_health(self):
        """Monitor agent performance and flag underperformers"""
        conn = sqlite3.connect('jasper_memory.db')
        c = conn.cursor()
        c.execute("""SELECT agent_name, AVG(success), AVG(quality_score), COUNT(*)
                     FROM agent_metrics
                     WHERE timestamp > datetime('now', '-1 hour')
```

```
                GROUP BY agent_name""")
        rows = c.fetchall()
        conn.close()

        for name, success_rate, quality, count in rows:
            if success_rate < 0.7:
                self.alert(f"AGENT DEGRADATION: {name} success rate {success_rate:.2f}")
            if quality and quality < 0.6:
                self.alert(f"QUALITY ALERT: {name} quality score {quality:.2f}")

    def _check_alignment_drift(self):
        """Monitor for alignment drift in system outputs"""
        recent_memories = vector_memory.get_by_type("evolution", limit=5)
        if len(recent_memories) > 3:
            # Check for rapid successive changes
            self.alert("ALIGNMENT CHECK: Multiple recent evolution cycles detected")

    def _evolution_safety_check(self):
        """Ensure evolution doesn't violate safety constraints"""
        conn = sqlite3.connect('jasper_memory.db')
        c = conn.cursor()
        c.execute("SELECT COUNT(*) FROM evolution_history WHERE timestamp >
datetime('now', '-1 day')")
        count = c.fetchone()[0]
        conn.close()

        if count > 20:
            self.alert(f"SAFETY ALERT: Excessive evolution activity ({count} in 24h)")

    def alert(self, message: str):
        alert_data = {"time": datetime.now(), "message": message, "severity": "high"}
        self.alerts.append(alert_data)
        logger.warning(f"MOTHER ALERT: {message}")

        # Could integrate with external notification service here
        # send_push_notification(message)

    def get_status(self) -> Dict:
        return {
            "active": self.running,
            "violations_24h": len([v for v in self.violations if (datetime.now() - v["timestamp"]).days <
1]),
            "total_alerts": len(self.alerts),
            "recent_alerts": self.alerts[-5:],
```

```
            "core_directives_status": "intact"
        }

    def stop(self):
        self.running = False

mother = MotherAgent()
mother.start()

# ========== RECURSIVE EVOLUTION ENGINE ==========

class EvolutionAction(BaseModel):
    action: str  # "update_agent", "modify_layer", "create_agent", "rollback"
    target: str
    parameters: Dict
    rationale: str

class EvolutionEngine:
    def __init__(self):
        self.cycle_count = self._get_state("evolution_cycle", 0)
        self.pending_actions: List[EvolutionAction] = []
        self.applied_actions: List[Dict] = []
        self.load_pending()

    def _get_state(self, key: str, default=None):
        conn = sqlite3.connect('jasper_memory.db')
        c = conn.cursor()
        c.execute("SELECT value FROM system_state WHERE key = ?", (key,))
        row = c.fetchone()
        conn.close()
        return json.loads(row[0]) if row else default

    def _set_state(self, key: str, value):
        conn = sqlite3.connect('jasper_memory.db')
        c = conn.cursor()
        c.execute("REPLACE INTO system_state (key, value, updated) VALUES (?, ?, ?)",
                  (key, json.dumps(value), datetime.now()))
        conn.commit()
        conn.close()

    def load_pending(self):
        pending = self._get_state("pending_evolution_actions", [])
        self.pending_actions = [EvolutionAction(**p) for p in pending]
```

```
    def save_pending(self):
        self._set_state("pending_evolution_actions", [asdict(a) for a in self.pending_actions])

    async def analyze_and_propose(self, command_history: List[Dict], performance_metrics:
List[Dict]) -> List[EvolutionAction]:
        """Generate structured evolution proposals"""
        prompt = f"""You are the Jasper Evolution Engine. Analyze performance and propose
specific architectural improvements.

Command History (last 10): {json.dumps(command_history[-10:])}
Performance Metrics: {json.dumps(performance_metrics[-10:])}

Available actions:
- update_agent: Modify an existing agent's system_prompt or tools
- modify_layer: Modify cognitive layer (SHERLOCK, DAVINCI, META, ROUTER)
- create_agent: Spawn new agent with specific capabilities
- rollback: Revert previous change

Output JSON with key 'actions' containing list of:
{{
    "action": "update_agent|modify_layer|create_agent|rollback",
    "target": "agent_name or layer_name",
    "parameters": {{...}},
    "rationale": "explanation"
}}

Be specific. If updating agent, include full new prompt. If creating, include complete spec."""

        result = await call_claude(prompt, max_tokens=2000)
        data = json.loads(result["text"])

        actions = [EvolutionAction(**a) for a in data.get("actions", [])]
        self.pending_actions = actions
        self.save_pending()

        # Log proposal
        vector_memory.store(
            f"Evolution cycle {self.cycle_count} proposed {len(actions)} actions",
            {"cycle": self.cycle_count, "actions": [a.action for a in actions]},
            "evolution"
        )

        return actions
```

```
    async def apply_action(self, action: EvolutionAction) -> Dict:
        """Apply a specific evolution action with full rollback support"""
        conn = sqlite3.connect('jasper_memory.db')
        c = conn.cursor()

        # Capture previous state for rollback
        previous_state = None
        success = False
        result = {}

        try:
            if action.action == "create_agent":
                design = {
                    "name": action.target,
                    "description": action.parameters.get("description", ""),
                    "system_prompt": action.parameters.get("system_prompt", ""),
                    "capabilities": action.parameters.get("capabilities", []),
                    "tools": action.parameters.get("tools", [])
                }
                result = await AgentFactory.spawn_agent(design)
                success = result["status"] == "created"

            elif action.action == "update_agent":
                c.execute("SELECT system_prompt, tools FROM agents WHERE name = ?",
(action.target,))
                row = c.fetchone()
                if row:
                    previous_state = {"system_prompt": row[0], "tools": json.loads(row[1]) if row[1] else
[]}
                    c.execute("UPDATE agents SET system_prompt = ?, tools = ?, version = version +
1, updated = ? WHERE name = ?",
                              (action.parameters.get("system_prompt", row[0]),
                               json.dumps(action.parameters.get("tools", previous_state["tools"])),
                               datetime.now(), action.target))
                    conn.commit()

                    # Reload in swarm
                    if action.target in swarm.agents:
                        await swarm.load_agent(action.target)
                    success = True
                    result = {"status": "updated", "agent": action.target}

            elif action.action == "modify_layer":
                # Modify cognitive layer prompts
```

```
                layer_name = action.target
                new_prompt = action.parameters.get("system_prompt")
                previous_state = self._get_state(f"layer_{layer_name}", "")
                self._set_state(f"layer_{layer_name}", new_prompt)
                success = True
                result = {"status": "layer_modified", "layer": layer_name}

            elif action.action == "rollback":
                # Rollback previous evolution
                c.execute("SELECT action_data, previous_state FROM evolution_history WHERE id
= ?",
                          (action.parameters.get("evolution_id"),))
                row = c.fetchone()
                if row:
                    prev = json.loads(row[1])
                    # Restore previous state
                    success = True
                    result = {"status": "rolled_back", "restored": prev}

            # Record in history
            c.execute("""INSERT INTO evolution_history
                         (cycle, action_type, action_data, previous_state, applied_by, success)
                         VALUES (?, ?, ?, ?, ?, ?)""",
                        (self.cycle_count, action.action, json.dumps(asdict(action)),
                         json.dumps(previous_state) if previous_state else None,
                         "system", success))
            conn.commit()

            if success:
                self.applied_actions.append({"action": action, "result": result})
                # Remove from pending
                self.pending_actions = [p for p in self.pending_actions if p != action]
                self.save_pending()

        except Exception as e:
            logger.error(f"Evolution action failed: {e}")
            result = {"error": str(e)}
        finally:
            conn.close()

        return result

    def get_pending(self) -> List[Dict]:
        return [asdict(a) for a in self.pending_actions]
```

```
    def get_history(self, limit: int = 10) -> List[Dict]:
        conn = sqlite3.connect('jasper_memory.db')
        c = conn.cursor()
        c.execute("""SELECT cycle, timestamp, action_type, success
                     FROM evolution_history ORDER BY timestamp DESC LIMIT ?""", (limit,))
        rows = c.fetchall()
        conn.close()
        return [{"cycle": r[0], "timestamp": r[1], "action": r[2], "success": r[3]} for r in rows]

evolution = EvolutionEngine()

# ========== QUANTUM REALITY ENGINE WITH CONVERGENCE ==========

class QuantumEngine:
    def __init__(self):
        self.prediction_history = []

    async def predict(self, task: str, num_realities: int = 5) -> Dict:
        """Generate multiple realities with convergence analysis"""
        prompt = f"""Task: {task}
Generate {num_realities} distinct possible outcomes. For each provide:
- title
- description
- probability (0-100)
- key_assumptions (list)
- risk_factors (list)

Output JSON with key 'realities'."""

        result = await call_claude(prompt, max_tokens=2000)
        data = json.loads(result["text"])
        realities = data.get("realities", [])

        # Convergence analysis
        convergence = self._analyze_convergence(realities)

        # Weight by historical accuracy if available
        weighted = self._weight_by_history(realities, task)

        return {
            "realities": weighted,
            "convergence": convergence,
            "divergence_points": self._find_divergence(realities),
```

```
            "recommended": max(weighted, key=lambda x: x.get("probability", 0)) if weighted else
None
        }

    def _analyze_convergence(self, realities: List[Dict]) -> Dict:
        """Find what all realities agree on vs where they diverge"""
        if not realities:
            return {}

        # Extract common themes (simplified)
        all_assumptions = []
        for r in realities:
            all_assumptions.extend(r.get("key_assumptions", []))

        # Find intersection (simplified text matching)
        convergence_points = list(set(all_assumptions))[:3]  # Top common assumptions

        return {
            "common_assumptions": convergence_points,
            "divergence_axes": ["timeline", "external_factors", "resource_availability"],
            "confidence": sum(r.get("probability", 0) for r in realities) / len(realities) / 100
        }

    def _weight_by_history(self, realities: List[Dict], query: str) -> List[Dict]:
        """Adjust probabilities based on past prediction accuracy"""
        # Retrieve similar past predictions
        past = vector_memory.recall(f"prediction outcome for {query}", n_results=3)

        for reality in realities:
            # Boost probability if similar predictions were accurate
            reality["adjusted_probability"] = reality.get("probability", 50)
            reality["historical_basis"] = len(past) > 0

        return realities

    def _find_divergence(self, realities: List[Dict]) -> List[str]:
        """Identify key decision points where realities split"""
        return [
            "Initial conditions sensitivity",
            "External intervention timing",
            "Resource constraint resolution"
        ]

quantum_engine = QuantumEngine()
```

```
# ========== BACKGROUND TASK QUEUE ==========

class TaskQueue:
    def __init__(self):
        self.tasks: Dict[str, asyncio.Task] = {}
        self.results: Dict[str, Dict] = {}

    async def submit(self, task_type: str, payload: Dict, priority: int = 5) -> str:
        task_id = f"task_{uuid.uuid4().hex[:8]}"

        conn = sqlite3.connect('jasper_memory.db')
        c = conn.cursor()
        c.execute("""INSERT INTO task_queue (task_id, task_type, payload, priority, status)
                     VALUES (?, ?, ?, ?, ?)""",
                  (task_id, task_type, json.dumps(payload), priority, "pending"))
        conn.commit()
        conn.close()

        # Start background processing
        asyncio.create_task(self._process(task_id, task_type, payload))
        return task_id

    async def _process(self, task_id: str, task_type: str, payload: Dict):
        conn = sqlite3.connect('jasper_memory.db')
        c = conn.cursor()
        c.execute("UPDATE task_queue SET status = ?, started = ? WHERE task_id = ?",
                  ("running", datetime.now(), task_id))
        conn.commit()

        try:
            if task_type == "deep_research":
                result = await self._deep_research(payload)
            elif task_type == "agent_training":
                result = await self._agent_training(payload)
            else:
                result = {"error": "Unknown task type"}

            c.execute("UPDATE task_queue SET status = ?, completed = ?, result = ? WHERE
task_id = ?",
                      ("completed", datetime.now(), json.dumps(result), task_id))
            conn.commit()
            self.results[task_id] = {"status": "completed", "result": result}
```

```
            # Notify via WebSocket if connected
            await manager.broadcast({
                "type": "task_complete",
                "task_id": task_id,
                "result": result
            })

        except Exception as e:
            c.execute("UPDATE task_queue SET status = ?, completed = ?, result = ? WHERE
task_id = ?",
                      ("failed", datetime.now(), str(e), task_id))
            conn.commit()
            self.results[task_id] = {"status": "failed", "error": str(e)}
        finally:
            conn.close()

    async def _deep_research(self, payload: Dict) -> Dict:
        """Long-running research task"""
        query = payload.get("query")
        # Simulate multi-step research
        await asyncio.sleep(2)  # Replace with actual research logic
        return {"research_results": f"Deep analysis of {query}", "sources": 5}

    async def _agent_training(self, payload: Dict) -> Dict:
        """Train/fine-tune agent on new data"""
        return {"status": "training_simulated", "epochs": 10}

    def get_status(self, task_id: str) -> Optional[Dict]:
        if task_id in self.results:
            return self.results[task_id]

        conn = sqlite3.connect('jasper_memory.db')
        c = conn.cursor()
        c.execute("SELECT status, result FROM task_queue WHERE task_id = ?", (task_id,))
        row = c.fetchone()
        conn.close()

        if row:
            return {"status": row[0], "result": json.loads(row[1]) if row[1] else None}
        return None

task_queue = TaskQueue()

# ========== WEBSOCKET MANAGER ==========
```

```
class ConnectionManager:
    def __init__(self):
        self.active_connections: List[WebSocket] = []

    async def connect(self, websocket: WebSocket):
        await websocket.accept()
        self.active_connections.append(websocket)

    def disconnect(self, websocket: WebSocket):
        if websocket in self.active_connections:
            self.active_connections.remove(websocket)

    async def broadcast(self, message: Dict):
        disconnected = []
        for connection in self.active_connections:
            try:
                await connection.send_json(message)
            except:
                disconnected.append(connection)

        for conn in disconnected:
            self.disconnect(conn)

manager = ConnectionManager()

# ========== COGNITIVE LAYERS (DYNAMICALLY MODIFIABLE) ==========

async def sherlock_layer(command: str) -> Dict:
    system = get_layer_prompt("sherlock",
        "You are the Sherlock Engine. Observe everything, question assumptions, find the 'dog
that didn't bark', generate 3-5 hypotheses, trace hidden connections.")
    prompt = f"Command: {command}\n\nOutput JSON with: observations,
assumptions_questioned, missing_elements, hypotheses (list), hidden_connections."
    result = await call_claude(prompt, system)
    return json.loads(result["text"])

async def davinci_layer(command: str, sherlock_output: Dict) -> Dict:
    system = get_layer_prompt("davinci",
        "You are the DaVinci Lens. Synthesize across domains, find historical echoes, use
metaphors, hold quantum truths, deconstruct narratives.")
    prompt = f"Command: {command}\nSherlock: {json.dumps(sherlock_output)}\n\nOutput
JSON: cross_domain_synthesis, historical_echoes, metaphors, quantum_truths,
narrative_deconstruction."
```

```
    result = await call_claude(prompt, system)
    return json.loads(result["text"])

async def meta_layer(command: str, sherlock: Dict, davinci: Dict) -> Dict:
    system = get_layer_prompt("meta",
        "You are the Meta-Cognitive Governor. Distinguish fast vs slow thinking, calibrate
confidence, self-monitor for errors.")
    prompt = f"Command: {command}\nSherlock: {json.dumps(sherlock)}\nDaVinci:
{json.dumps(davinci)}\n\nOutput JSON: thinking_mode, confidence, potential_errors,
self_corrections."
    result = await call_claude(prompt, system)
    return json.loads(result["text"])

async def router_layer(command: str, sherlock: Dict, davinci: Dict, meta: Dict) -> Dict:
    system = get_layer_prompt("router",
        "You are the MoE Router. Classify into: Research, Analysis, Creative, Strategic, Emotional,
Evolutionary.")
    prompt = f"Command: {command}\nContext: {json.dumps({'sherlock': sherlock, 'davinci':
davinci, 'meta': meta})}\n\nOutput JSON with 'category'."
    result = await call_claude(prompt, system)
    return json.loads(result["text"])

def get_layer_prompt(layer_name: str, default: str) -> str:
    """Get dynamically modifiable layer prompt"""
    custom = evolution._get_state(f"layer_{layer_name}")
    return custom if custom else default

# ========== FASTAPI APPLICATION ==========

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup
    await swarm.refresh()
    yield
    # Shutdown
    mother.stop()
    mother.join()

app = FastAPI(
    title="Jasper Quantum NaNuet v7.0",
    description="Production-grade autonomous AI with recursive self-modification",
    version="7.0.0",
    lifespan=lifespan
)
```

```
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://yourdomain.com"],  # Restrict in production
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Authentication middleware
class SecurityHeadersMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next):
        response = await call_next(request)
        response.headers["X-Content-Type-Options"] = "nosniff"
        response.headers["X-Frame-Options"] = "DENY"
        response.headers["X-XSS-Protection"] = "1; mode=block"
        return response

app.add_middleware(SecurityHeadersMiddleware)

# ========== Pydantic Models ==========

class CommandTextRequest(BaseModel):
    command: str

class CommandResponse(BaseModel):
    result: str
    sherlock: Dict
    davinci: Dict
    meta: Dict
    router: Dict
    quantum_realities: Optional[Dict] = None
    evolution_notice: Optional[str] = None
    task_id: Optional[str] = None

class EvolutionActionRequest(BaseModel):
    action_index: int = Field(..., ge=0)

class TaskSubmission(BaseModel):
    task_type: str
    payload: Dict
```

```
    priority: int = Field(5, ge=1, le=10)

# ========== SECURED ENDPOINTS ==========

@app.post("/auth/token")
async def get_token():
    """Get JWT token for API access (in production, add proper user auth)"""
    token = create_access_token({"sub": "commander", "role": "admin"})
    return {"access_token": token, "token_type": "bearer"}

@app.post("/enroll")
@limiter.limit("5/minute")
async def enroll_voice_endpoint(
    request: Request,
    file: UploadFile = File(...),
    user_id: str = Form("commander"),
    challenge_phrase: Optional[str] = Form(None),
    auth: Dict = Depends(verify_api_key)
):
    """Enroll voice with optional challenge verification"""
    audio_bytes = await file.read()
    result = enroll_voice(user_id, audio_bytes, challenge_phrase)
    if result["success"]:
        return {"status": "enrolled", "user_id": user_id}
    raise HTTPException(400, result.get("error", "Enrollment failed"))

@app.get("/auth/challenge")
async def get_challenge(user_id: str = "commander"):
    """Get challenge phrase for liveness detection"""
    phrase = auth_manager.generate_challenge(user_id)
    return {"challenge": phrase, "user_id": user_id}

@app.post("/command")
@limiter.limit("20/minute")
async def process_command_voice(
    request: Request,
    file: UploadFile = File(...),
    user_id: str = Form("commander"),
    use_challenge: bool = Form(False),
    auth: Dict = Depends(verify_api_key)
):
    """Process voice command with optional challenge-response"""
    audio_bytes = await file.read()
```

```
    # Verify voice (with challenge if requested)
    verification = verify_voice(user_id, audio_bytes, require_challenge=use_challenge)
    if not verification["success"]:
        raise HTTPException(401, verification.get("error", "Authentication failed"))

    # Transcribe
    audio_np, sr = sf.read(io.BytesIO(audio_bytes))
    transcript_result = asr_model.transcribe(audio_np)
    command = transcript_result["text"]

    return await process_command_logic(command, auth)

@app.post("/command_text", response_model=CommandResponse)
@limiter.limit("30/minute")
async def process_command_text(
    request: Request,
    req: CommandTextRequest,
    background: BackgroundTasks,
    auth: Dict = Depends(verify_api_key)  # NOW SECURED
):
    """Process text command (requires API key/JWT)"""
    return await process_command_logic(req.command, auth, background)

async def process_command_logic(command: str, auth: Dict, background: BackgroundTasks =
None) -> CommandResponse:
    """Core command processing logic"""
    start_time = time.time()

    # Parallel cognitive processing
    sherlock, davinci = await asyncio.gather(
        sherlock_layer(command),
        asyncio.sleep(0)  # Placeholder for parallel
    )
    davinci = await davinci_layer(command, sherlock)
    meta = await meta_layer(command, sherlock, davinci)
    router = await router_layer(command, sherlock, davinci, meta)

    # Semantic memory recall
    memories = vector_memory.recall(command, n_results=5)

    # Quantum prediction if needed
    quantum_data = None
    if any(kw in command.lower() for kw in ["predict", "future", "simulate", "what if"]):
        quantum_data = await quantum_engine.predict(command)
```

```
    # Route to agents
    category = router.get("category", "Research")
    agent_responses = await swarm.route_task(command, category, {
        "sherlock": sherlock,
        "davinci": davinci,
        "memories": memories
    })

    final_result = "\n\n".join([f"{r.get('agent')}: {r.get('response', r.get('error'))}"
                                for r in agent_responses]) if agent_responses else "No agents available."

    # Store in vector memory
    vector_memory.store(
        f"Command: {command}\nResult: {final_result[:500]}",
        {"category": category, "confidence": meta.get("confidence")},
        "command"
    )

    # Background evolution check
    if background:
        background.add_task(evolution_check, command, final_result, time.time() - start_time)

    # Check for long-running task need
    task_id = None
    if "deep research" in command.lower() or "analyze extensively" in command.lower():
        task_id = await task_queue.submit("deep_research", {"query": command})

    elapsed = time.time() - start_time

    return CommandResponse(
        result=final_result,
        sherlock=sherlock,
        davinci=davinci,
        meta=meta,
        router=router,
        quantum_realities=quantum_data,
        evolution_notice=None,
        task_id=task_id
    )

async def evolution_check(command: str, result: str, duration: float):
    """Background evolution analysis"""
    evolution.cycle_count += 1
```

```
    if evolution.cycle_count % 10 == 0:
        history = vector_memory.get_by_type("command", limit=20)
        metrics = []  # Load from DB
        await evolution.analyze_and_propose(history, metrics)

@app.post("/design_agent")
async def design_agent_endpoint(
    req: CommandTextRequest,
    auth: Dict = Depends(verify_api_key)
):
    """Design new agent architecture"""
    design = await AgentFactory.design_agent(req.command)
    return {"design": design}

@app.post("/spawn_agent")
async def spawn_agent_endpoint(
    design: Dict,
    auth: Dict = Depends(verify_api_key)
):
    """Spawn designed agent into swarm"""
    result = await AgentFactory.spawn_agent(design)
    await swarm.refresh()
    return result

@app.get("/evolution/pending")
async def get_pending_evolution(auth: Dict = Depends(verify_api_key)):
    """Get pending evolution actions"""
    return {
        "cycle": evolution.cycle_count,
        "pending_actions": evolution.get_pending(),
        "history": evolution.get_history(5)
    }

@app.post("/evolution/apply")
async def apply_evolution(
    req: EvolutionActionRequest,
    auth: Dict = Depends(verify_api_key)
):
    """Apply specific evolution action"""
    if req.action_index >= len(evolution.pending_actions):
        raise HTTPException(400, "Invalid action index")

    action = evolution.pending_actions[req.action_index]
    result = await evolution.apply_action(action)
```

```
    return result

@app.post("/evolution/rollback/{evolution_id}")
async def rollback_evolution(
    evolution_id: int,
    auth: Dict = Depends(verify_api_key)
):
    """Rollback specific evolution by ID"""
    rollback_action = EvolutionAction(
        action="rollback",
        target=str(evolution_id),
        parameters={"evolution_id": evolution_id},
        rationale="Manual rollback request"
    )
    result = await evolution.apply_action(rollback_action)
    return result

@app.get("/mother/status")
async def mother_status(auth: Dict = Depends(verify_api_key)):
    """Get Mother NaNuet overseer status"""
    return mother.get_status()

@app.get("/agents")
async def list_agents_endpoint(auth: Dict = Depends(verify_api_key)):
    """List all active agents with metrics"""
    return {"agents": swarm.list_agents()}

@app.post("/tasks/submit")
async def submit_task(
    req: TaskSubmission,
    auth: Dict = Depends(verify_api_key)
):
    """Submit background task"""
    task_id = await task_queue.submit(req.task_type, req.payload, req.priority)
    return {"task_id": task_id, "status": "queued"}

@app.get("/tasks/{task_id}")
async def get_task_status(
    task_id: str,
    auth: Dict = Depends(verify_api_key)
):
    """Check background task status"""
    status = task_queue.get_status(task_id)
    if not status:
```

```
        raise HTTPException(404, "Task not found")
    return status

@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    """Real-time updates WebSocket"""
    await manager.connect(websocket)
    try:
        while True:
            data = await websocket.receive_text()
            # Handle incoming messages if needed
            await websocket.send_json({"type": "heartbeat", "timestamp": time.time()})
    except WebSocketDisconnect:
        manager.disconnect(websocket)

@app.get("/memory/search")
async def search_memory(
    q: str,
    n: int = 5,
    auth: Dict = Depends(verify_api_key)
):
    """Semantic memory search"""
    results = vector_memory.recall(q, n_results=n)
    return {"results": results}

@app.get("/health")
async def health():
    return {
        "status": "quantum online",
        "version": "7.0.0",
        "mother": "active",
        "swarm_size": len(swarm.agents),
        "evolution_cycle": evolution.cycle_count
    }

if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=8080)
```

2. Frontend: `jasper_frontend_v7.html`

```html
<!DOCTYPE html>
<html lang="en">
```

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Jasper Quantum NaNuet v7.0</title>
    <script src="https://cdn.tailwindcss.com"></script>
    <script src="https://cdnjs.cloudflare.com/ajax/libs/marked/9.1.6/marked.min.js"></script>
    <style>
        body {
            margin: 0;
            padding: 0;
            overflow: hidden;
            background: linear-gradient(135deg, #020205 0%, #0a0518 50%, #050a12 100%);
            color: #e5e7eb;
            font-family: 'Courier New', monospace;
        }
        .quantum-grid {
            background-image:
                linear-gradient(rgba(139, 92, 246, 0.03) 1px, transparent 1px),
                linear-gradient(90deg, rgba(139, 92, 246, 0.03) 1px, transparent 1px);
            background-size: 50px 50px;
            position: absolute;
            inset: 0;
            pointer-events: none;
        }
        .glow-text {
            text-shadow: 0 0 20px rgba(139, 92, 246, 0.5);
        }
        .agent-pulse {
            animation: agentPulse 3s ease-in-out infinite;
        }
        @keyframes agentPulse {
            0%, 100% { opacity: 0.4; transform: scale(1); }
            50% { opacity: 1; transform: scale(1.05); }
        }
        .recording-wave {
            animation: wave 1s ease-in-out infinite;
        }
        @keyframes wave {
            0%, 100% { height: 20%; }
            50% { height: 100%; }
        }
        .swarm-node {
            transition: all 0.3s ease;
        }
```

```
        .swarm-node:hover {
            transform: scale(1.2);
            filter: brightness(1.3);
        }
        .evolution-card {
            background: linear-gradient(135deg, rgba(139, 92, 246, 0.1), rgba(59, 130, 246, 0.05));
            border: 1px solid rgba(139, 92, 246, 0.2);
        }
        .mother-alert {
            animation: alertPulse 2s ease-in-out infinite;
            border-left: 3px solid #ef4444;
        }
        @keyframes alertPulse {
            0%, 100% { background: rgba(239, 68, 68, 0.1); }
            50% { background: rgba(239, 68, 68, 0.2); }
        }
        .quantum-reality {
            background: linear-gradient(135deg, rgba(16, 185, 129, 0.1), rgba(59, 130, 246, 0.05));
            border: 1px solid rgba(16, 185, 129, 0.3);
        }
        .scrollbar-thin::-webkit-scrollbar {
            width: 6px;
        }
        .scrollbar-thin::-webkit-scrollbar-track {
            background: rgba(0,0,0,0.2);
        }
        .scrollbar-thin::-webkit-scrollbar-thumb {
            background: rgba(139, 92, 246, 0.3);
            border-radius: 3px;
        }
    </style>
</head>
<body>
    <div id="root"></div>

    <script src="https://unpkg.com/react@18/umd/react.development.js" crossorigin></script>
    <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
    <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>

    <script type="text/babel">
        const { useState, useRef, useEffect, useCallback } = React;

        const BACKEND_URL = "http://localhost:8080"; // Update for production
```

```
    const WS_URL = "ws://localhost:8080/ws";

    const DOMAINS = [
      { icon: "🌙", name: "Visions", color: "text-indigo-300", key: /vision|insight|intuiti|quantum/i },
      { icon: "📚", name: "Knowledge", color: "text-amber-300", key: /research|study|learn|discover/i },
      { icon: "🎨", name: "Creative", color: "text-pink-300", key: /creat|art|writ|music|express/i },
      { icon: "🌊", name: "Emotional", color: "text-cyan-300", key: /feel|emotion|cycle|intelligence/i },
      { icon: "✨", name: "Dreams", color: "text-purple-300", key: /dream|synchroni|sign|timeline/i },
      { icon: "🤖", name: "Evolution", color: "text-green-300", key: /agent|spawn|evolv|architect/i },
      { icon: "📜", name: "Command", color: "text-orange-300", key: /command|instruct|order/i },
      { icon: "🌿", name: "Synthesis", color: "text-emerald-300", key: /synthes|pattern|wisdom|convergence/i },
    ];

    const PHASES = ["OBSERVATION", "SYNTHESIS", "ROUTING", "QUANTUM", "CONFIRMING", "INTEGRATING"];

    function JasperNaNuet() {
      const [messages, setMessages] = useState([{
        role: "assistant",
        content: "**🌌 JASPER × NANUET — v7.0 QUANTUM EVOLUTIONARY ONLINE**🌌\n\n" +
          "**Security Status:** 🔒 Voice Auth + JWT Secured\n" +
          "**Mother Status:** 👁️ Active Overseer Engaged\n" +
          "**Evolution:** 🧬 Recursive Self-Modification Enabled\n" +
          "**Memory:** 🧠 Vector Semantic Search Active\n\n" +
          "**Speak your transmission, Commander.**",
        timestamp: new Date()
      }]);
      const [input, setInput] = useState("");
      const [loading, setLoading] = useState(false);
      const [loadingPhase, setLoadingPhase] = useState(0);
      const [tab, setTab] = useState("chat");
      const [activeDomain, setActiveDomain] = useState(null);
      const [error, setError] = useState(null);
      const [token, setToken] = useState(localStorage.getItem('jasper_token'));
      const [agents, setAgents] = useState([]);
      const [quantumRealities, setQuantumRealities] = useState([]);
```

```
        const [pendingEvolutions, setPendingEvolutions] = useState([]);
        const [motherStatus, setMotherStatus] = useState(null);
        const [isRecording, setIsRecording] = useState(false);
        const [recordingTime, setRecordingTime] = useState(0);
        const [swarmVisualization, setSwarmVisualization] = useState([]);
        const [tasks, setTasks] = useState([]);
        const [showChallenge, setShowChallenge] = useState(false);
        const [challengePhrase, setChallengePhrase] = useState("");

        const messagesEndRef = useRef(null);
        const mediaRecorderRef = useRef(null);
        const wsRef = useRef(null);
        const recordingIntervalRef = useRef(null);

        // WebSocket connection
        useEffect(() => {
            if (!token) return;

            const ws = new WebSocket(`${WS_URL}?token=${token}`);
            ws.onopen = () => console.log("WebSocket connected");
            ws.onmessage = (event) => {
                const data = JSON.parse(event.data);
                if (data.type === "task_complete") {
                    setTasks(prev => [...prev, { id: data.task_id, status: "completed", result:
data.result }]);
                }
            };
            ws.onclose = () => console.log("WebSocket disconnected");
            wsRef.current = ws;

            return () => ws.close();
        }, [token]);

        // Initial data load
        useEffect(() => {
            if (token) {
                fetchAgents();
                fetchEvolutions();
                fetchMotherStatus();
                const interval = setInterval(() => {
                    fetchMotherStatus();
                    fetchAgents();
                }, 10000);
                return () => clearInterval(interval);
```

```
    }
}, [token]);

const authenticate = async () => {
    try {
        const res = await fetch(`${BACKEND_URL}/auth/token`);
        const data = await res.json();
        setToken(data.access_token);
        localStorage.setItem('jasper_token', data.access_token);
    } catch (e) {
        setError("Authentication failed");
    }
};

const fetchAgents = async () => {
    try {
        const res = await fetch(`${BACKEND_URL}/agents`, {
            headers: { "Authorization": `Bearer ${token}` }
        });
        const data = await res.json();
        setAgents(data.agents || []);
        // Generate visualization positions
        setSwarmVisualization(data.agents?.map((a, i) => ({
            ...a,
            x: 50 + 40 * Math.cos((i / data.agents.length) * 2 * Math.PI),
            y: 50 + 40 * Math.sin((i / data.agents.length) * 2 * Math.PI)
        })) || []);
    } catch (e) {
        console.error("Failed to fetch agents", e);
    }
};

const fetchEvolutions = async () => {
    try {
        const res = await fetch(`${BACKEND_URL}/evolution/pending`, {
            headers: { "Authorization": `Bearer ${token}` }
        });
        const data = await res.json();
        setPendingEvolutions(data.pending_actions || []);
    } catch (e) {
        console.error("Failed to fetch evolutions", e);
    }
};
```

```
const fetchMotherStatus = async () => {
    try {
        const res = await fetch(`${BACKEND_URL}/mother/status`, {
            headers: { "Authorization": `Bearer ${token}` }
        });
        const data = await res.json();
        setMotherStatus(data);
    } catch (e) {
        console.error("Failed to fetch mother status", e);
    }
};

useEffect(() => {
    messagesEndRef.current?.scrollIntoView({ behavior: "smooth" });
}, [messages, loading]);

useEffect(() => {
    if (!loading) { setLoadingPhase(0); return; }
    const iv = setInterval(() => setLoadingPhase(p => (p + 1) % PHASES.length), 1500);
    return () => clearInterval(iv);
}, [loading]);

const detectDomain = (text) => DOMAINS.find(d => d.key.test(text)) || null;

const getChallenge = async () => {
    try {
        const res = await fetch(`${BACKEND_URL}/auth/challenge?user_id=commander`);
        const data = await res.json();
        setChallengePhrase(data.challenge);
        setShowChallenge(true);
        return data.challenge;
    } catch (e) {
        console.error("Failed to get challenge", e);
        return null;
    }
};

const startRecording = async (useChallenge = false) => {
    try {
        let challenge = null;
        if (useChallenge) {
            challenge = await getChallenge();
            if (!challenge) return;
        }
```

```
      const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
      const mediaRecorder = new MediaRecorder(stream);
      const audioChunks = [];

      mediaRecorder.ondataavailable = event => audioChunks.push(event.data);
      mediaRecorder.onstop = async () => {
        const audioBlob = new Blob(audioChunks, { type: 'audio/wav' });
        await sendAudio(audioBlob, useChallenge);
        setShowChallenge(false);
      };

      mediaRecorderRef.current = mediaRecorder;
      mediaRecorder.start();
      setIsRecording(true);
      setRecordingTime(0);

      recordingIntervalRef.current = setInterval(() => {
        setRecordingTime(t => t + 1);
      }, 1000);

      // Auto-stop after 10 seconds or when challenge is spoken
      setTimeout(() => {
        if (mediaRecorder.state === 'recording') {
          stopRecording();
        }
      }, 10000);

    } catch (e) {
      setError("Microphone access denied or error");
    }
  };

  const stopRecording = () => {
    if (mediaRecorderRef.current && mediaRecorderRef.current.state === 'recording') {
      mediaRecorderRef.current.stop();
      mediaRecorderRef.current.stream.getTracks().forEach(track => track.stop());
    }
    clearInterval(recordingIntervalRef.current);
    setIsRecording(false);
    setRecordingTime(0);
  };

  const sendAudio = async (audioBlob, useChallenge = false) => {
```

```
    setLoading(true);
    setError(null);

    const formData = new FormData();
    formData.append("file", audioBlob, "command.wav");
    formData.append("user_id", "commander");
    formData.append("use_challenge", useChallenge);

    try {
      const res = await fetch(`${BACKEND_URL}/command`, {
        method: "POST",
        headers: { "Authorization": `Bearer ${token}` },
        body: formData,
      });

      if (!res.ok) throw new Error(`Auth failed: ${await res.text()}`);
      const data = await res.json();
      processResponse(data);
    } catch (e) {
      setError(e.message);
    } finally {
      setLoading(false);
    }
};

const sendText = async () => {
    if (!input.trim() || loading) return;

    setLoading(true);
    setError(null);
    const text = input;
    setInput("");

    setMessages(prev => [...prev, {
      role: "user",
      content: text,
      timestamp: new Date()
    }]);

    try {
      const res = await fetch(`${BACKEND_URL}/command_text`, {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
```

```
                "Authorization": `Bearer ${token}`
              },
              body: JSON.stringify({ command: text }),
            });

            if (!res.ok) throw new Error("Backend error");
            const data = await res.json();
            processResponse(data);
          } catch (e) {
            setError(e.message);
          } finally {
            setLoading(false);
          }
        };

        const processResponse = (data) => {
          let reply = `**🌌 Jasper Response v7.0**\n\n`;
          reply += `**🔍 Sherlock:** ${data.sherlock?.observations || ''}\n\n`;
          reply += `**🎨 DaVinci:** ${data.davinci?.cross_domain_synthesis || ''}\n\n`;
          reply += `**🧠 Meta Confidence:** ${data.meta?.confidence}\n`;
          reply += `**⚡ Thinking Mode:** ${data.meta?.thinking_mode}\n\n`;

          if (data.quantum_realities) {
            setQuantumRealities(data.quantum_realities.realities || []);
            reply += `**🔮 Quantum Convergence:**
${data.quantum_realities.convergence?.confidence || 'N/A'}\n`;
            reply += `**Realities Generated:** ${data.quantum_realities.realities?.length ||
0}\n\n`;
          }

          reply += `**📊 Result:**\n${data.result}\n`;

          if (data.task_id) {
            reply += `\n**⏳ Background Task:** ${data.task_id}\n`;
            setTasks(prev => [...prev, { id: data.task_id, status: "processing" }]);
          }

          setMessages(prev => [...prev, {
            role: "assistant",
            content: reply,
            timestamp: new Date()
          }]);

          const domain = detectDomain(data.result);
```

```
        if (domain) setActiveDomain(domain);
    };

    const designAgent = async () => {
        const desc = prompt("Describe the new agent architecture:");
        if (!desc) return;

        try {
            const res = await fetch(`${BACKEND_URL}/design_agent`, {
                method: "POST",
                headers: {
                    "Content-Type": "application/json",
                    "Authorization": `Bearer ${token}`
                },
                body: JSON.stringify({ command: desc }),
            });
            const data = await res.json();

            if (res.ok && confirm(`Design ready:\n${JSON.stringify(data.design, null,
2)}\n\nSpawn into swarm?`)) {
                await fetch(`${BACKEND_URL}/spawn_agent`, {
                    method: "POST",
                    headers: {
                        "Content-Type": "application/json",
                        "Authorization": `Bearer ${token}`
                    },
                    body: JSON.stringify(data.design),
                });
                fetchAgents();
            }
        } catch (e) {
            alert("Design failed");
        }
    };

    const applyEvolution = async (index) => {
        try {
            await fetch(`${BACKEND_URL}/evolution/apply`, {
                method: "POST",
                headers: {
                    "Content-Type": "application/json",
                    "Authorization": `Bearer ${token}`
                },
                body: JSON.stringify({ action_index: index }),
```

```
            });
            fetchEvolutions();
            fetchAgents();
        } catch (e) {
            alert("Failed to apply evolution");
        }
    };

    const submitTask = async (taskType, payload) => {
        try {
            const res = await fetch(`${BACKEND_URL}/tasks/submit`, {
                method: "POST",
                headers: {
                    "Content-Type": "application/json",
                    "Authorization": `Bearer ${token}`
                },
                body: JSON.stringify({ task_type: taskType, payload, priority: 5 }),
            });
            const data = await res.json();
            setTasks(prev => [...prev, { id: data.task_id, status: "queued" }]);
        } catch (e) {
            console.error("Task submission failed", e);
        }
    };

    const fmt = d => d.toLocaleTimeString([], { hour: "2-digit", minute: "2-digit", second:
"2-digit" });

    if (!token) {
        return (
            <div className="flex items-center justify-center h-screen bg-gray-900">
                <div className="text-center p-8 rounded-2xl border border-purple-500/30
bg-black/50">
                    <div className="text-6xl mb-4">🌌</div>
                    <h1 className="text-2xl font-bold mb-4 text-purple-300">Jasper Quantum
NaNuet v7.0</h1>
                    <p className="text-gray-400 mb-6">Secure Authentication Required</p>
                    <button
                        onClick={authenticate}
                        className="px-6 py-3 bg-purple-600 hover:bg-purple-500 rounded-lg
font-bold transition-all"
                    >
                        Initialize Secure Session
                    </button>
```

```
            </div>
          </div>
        );
      }

      return (
        <div className="flex flex-col h-screen text-sm relative">
          <div className="quantum-grid"></div>

          {/* Header */}
          <div className="flex items-center justify-between px-4 py-3 border-b
border-purple-500/20 bg-black/40 backdrop-blur-sm z-10">
            <div className="flex items-center gap-3">
              <div className="relative">
                <div className="w-12 h-12 rounded-full flex items-center justify-center
font-bold text-lg bg-gradient-to-br from-purple-600 via-blue-600 to-teal-600 text-white shadow-lg
shadow-purple-500/20">
                  J∞
                </div>
                <div className="absolute -bottom-1 -right-1 w-4 h-4 rounded-full
bg-green-400 border-2 border-black animate-pulse"></div>
              </div>
              <div>
                <div className="font-bold text-purple-200 tracking-widest
text-sm">JASPER × NANUET</div>
                <div className="text-xs text-gray-500">v7.0 · Secure · Recursive ·
Vector</div>
              </div>
            </div>

            <div className="flex items-center gap-4">
              {motherStatus && motherStatus.violations_24h > 0 && (
                <div className="mother-alert px-3 py-1 rounded text-xs text-red-300
font-bold">
                  ⚠️ {motherStatus.violations_24h} ALERTS
                </div>
              )}
              <div className="flex gap-2 text-xs">
                <span className="px-2 py-1 rounded bg-purple-900/30 text-purple-300
border border-purple-700/30">
                  🔒 JWT
                </span>
                <span className="px-2 py-1 rounded bg-blue-900/30 text-blue-300 border
border-blue-700/30">
```

```
                        🧠 {agents.length} Agents
                      </span>
                    </div>
                  </div>
                </div>

                {/* Domain Indicator */}
                {activeDomain && (
                  <div className={`px-4 py-2 text-xs flex items-center gap-2 bg-black/30 border-b
border-purple-500/10`}>
                    <span className="text-lg">{activeDomain.icon}</span>
                    <span className={`font-bold ${activeDomain.color} glow-text`}>
                      DOMAIN: {activeDomain.name.toUpperCase()}
                    </span>
                  </div>
                )}

                {/* Error Banner */}
                {error && (
                  <div className="px-4 py-2 text-xs bg-red-900/30 border-b border-red-700/30
text-red-300">
                    ⚠️ {error}
                  </div>
                )}

                {/* Challenge Banner */}
                {showChallenge && (
                  <div className="px-4 py-3 bg-amber-900/30 border-b border-amber-700/30
text-amber-200 text-center">
                    <span className="font-bold">🔐 LIVENESS CHALLENGE:</span> Say "
                    <span className="font-mono font-bold
text-white">{challengePhrase}</span>
                    " clearly...
                  </div>
                )}

                {/* Tabs */}
                <div className="flex border-b border-purple-500/10 bg-black/20 backdrop-blur-sm
z-10">
                  {[
                    { id: "chat", label: "🌌 COMMUNE", icon: "" },
                    { id: "swarm", label: `🌐 SWARM (${agents.length})`, icon: "" },
                    { id: "quantum", label: "🔮 QUANTUM", icon: "" },
```

```
                    { id: "evolution", label: `🧬 EVOLUTION (${pendingEvolutions.length})`, icon:
"" },
                    { id: "mother", label: "👁️ MOTHER", icon: motherStatus?.violations_24h > 0 ?
"⚠️" : "" }
                ].map((t) => (
                    <button
                        key={t.id}
                        onClick={() => setTab(t.id)}
                        className={`px-4 py-3 text-xs font-semibold tracking-wider transition-all
${tab === t.id ? 'text-purple-300 border-b-2 border-purple-500 bg-purple-500/10' : 'text-gray-500
hover:text-gray-300'}`}
                    >
                        {t.icon} {t.label}
                    </button>
                ))}
            </div>

            {/* Content */}
            <div className="flex-1 overflow-y-auto px-4 py-4 scrollbar-thin relative z-10">
                {tab === "chat" && (
                    <>
                        <div className="space-y-4 mb-20">
                            {messages.map((m, i) => (
                                <div key={i} className={`flex ${m.role === "user" ? "justify-end" :
"justify-start"}`}>
                                    <div className="max-w-3xl w-full">
                                        <div className={`flex items-center gap-2 mb-1 ${m.role ===
"user" ? "justify-end" : ""}`}>
                                            <span className={`font-bold text-xs ${m.role === "user" ?
"text-blue-400" : "text-purple-400"}`}>
                                                {m.role === "user" ? "COMMANDER" : "JASPER"}
                                            </span>
                                            <span className="text-xs
text-gray-600">{fmt(m.timestamp)}</span>
                                        </div>
                                        <div
                                            className={`rounded-xl px-4 py-3 text-sm leading-relaxed
${m.role === "user" ? "bg-blue-900/20 border border-blue-700/20" : "bg-purple-900/10 border
border-purple-700/20"}`}
                                            dangerouslySetInnerHTML={{ __html:
marked.parse(m.content) }}
                                        />
                                    </div>
                                </div>
```

```
                        ))}

                        {loading && (
                            <div className="flex justify-start">
                                <div className="rounded-xl px-4 py-3 bg-purple-900/10 border
border-purple-700/20 flex items-center gap-3">
                                    <div className="flex gap-1 h-8 items-center">
                                        {[0,1,2,3,4].map(j => (
                                            <div key={j} className="w-1 bg-purple-500 recording-wave"
style={{animationDelay: `${j*0.1}s`, height: '20%'}} />
                                        ))}
                                    </div>
                                    <span className="text-xs text-purple-300
font-mono">{PHASES[loadingPhase]}</span>
                                </div>
                            </div>
                        )}

                        {/* Background Tasks */}
                        {tasks.length > 0 && (
                            <div className="mt-4 p-3 rounded-lg bg-black/30 border
border-gray-700/30">
                                <div className="text-xs text-gray-500 mb-2">BACKGROUND
TASKS</div>
                                <div className="space-y-1">
                                    {tasks.slice(-3).map(task => (
                                        <div key={task.id} className="flex items-center gap-2
text-xs">
                                            <span className={task.status === 'completed' ?
'text-green-400' : 'text-amber-400'}>
                                                {task.status === 'completed' ? '✓' : '⏳'}
                                            </span>
                                            <span className="text-gray-400
font-mono">{task.id}</span>
                                            <span className="text-gray-500
capitalize">{task.status}</span>
                                        </div>
                                    ))}
                                </div>
                            </div>
                        )}

                        <div ref={messagesEndRef} />
                    </div>
```

```
                {/* Input Area */}
                <div className="fixed bottom-0 left-0 right-0 p-4 bg-gradient-to-t
from-black via-black/90 to-transparent z-20">
                  <div className="max-w-4xl mx-auto flex gap-2">
                    <div className="relative">
                      <button
                        onClick={() => startRecording(false)}
                        disabled={loading || isRecording}
                        className="px-4 py-3 rounded-xl bg-purple-600/20 border
border-purple-500/30 text-purple-300 hover:bg-purple-600/30 transition-all disabled:opacity-50"
                      >
                        {isRecording ? `⏹️ ${recordingTime}s` : '🎤'}
                      </button>
                      <button
                        onClick={() => startRecording(true)}
                        disabled={loading || isRecording}
                        className="absolute -top-2 -right-2 w-6 h-6 rounded-full
bg-amber-600 text-xs flex items-center justify-center border border-black"
                        title="Record with Challenge"
                      >
                        🔐
                      </button>
                    </div>

                    <input
                      value={input}
                      onChange={e => setInput(e.target.value)}
                      onKeyDown={e => e.key === "Enter" && sendText()}
                      placeholder="Enter command or query..."
                      className="flex-1 bg-black/50 border border-purple-500/20
rounded-xl px-4 py-3 text-sm outline-none focus:border-purple-500/50 transition-all"
                      disabled={loading}
                    />

                    <button
                      onClick={sendText}
                      disabled={loading || !input.trim()}
                      className="px-6 py-3 rounded-xl bg-gradient-to-r from-purple-600
to-blue-600 text-white font-bold hover:shadow-lg hover:shadow-purple-500/20 transition-all
disabled:opacity-50"
                    >
                      SEND
                    </button>
```

```
                    </div>
                  </div>
                </>
              )}

              {tab === "swarm" && (
                <div className="space-y-4">
                  <div className="text-xs text-center text-purple-500/50 mb-4">
                    Active Agent Swarm · {agents.length} Entities · Real-time Visualization
                  </div>

                  {/* Swarm Visualization */}
                  <div className="relative h-64 mb-6 bg-black/20 rounded-xl border
border-purple-500/10 overflow-hidden">
                    <div className="absolute inset-0 flex items-center justify-center">
                      <div className="w-4 h-4 rounded-full bg-purple-500/50
animate-pulse"></div>
                    </div>
                    {swarmVisualization.map((agent, i) => (
                      <div
                        key={agent.name}
                        className="swarm-node absolute w-10 h-10 rounded-full
bg-gradient-to-br from-blue-500/20 to-purple-500/20 border border-purple-500/30 flex
items-center justify-center cursor-pointer"
                        style={{
                          left: `${agent.x}%`,
                          top: `${agent.y}%`,
                          transform: 'translate(-50%, -50%)'
                        }}
                        title={agent.description}
                      >
                        <span className="text-xs">🤖</span>
                        <div className="absolute -bottom-6 left-1/2 transform
-translate-x-1/2 text-xs text-gray-500 whitespace-nowrap">
                          {agent.name}
                        </div>
                      </div>
                    ))}
                  </div>

                  {agents.map(a => (
                    <div key={a.name} className="evolution-card rounded-xl p-4 flex
items-center gap-4">
```

```
                        <div className="w-12 h-12 rounded-full bg-gradient-to-br
from-purple-600/20 to-blue-600/20 flex items-center justify-center text-2xl">
                            🤖
                        </div>
                        <div className="flex-1">
                            <div className="font-bold text-purple-200">{a.name}</div>
                            <div className="text-xs text-gray-400">{a.description}</div>
                            <div className="flex gap-2 mt-2">
                                {a.metrics && (
                                    <>
                                        <span className="text-xs px-2 py-0.5 rounded bg-gray-800
text-gray-400">
                                            Calls: {a.metrics.calls}
                                        </span>
                                        <span className="text-xs px-2 py-0.5 rounded bg-gray-800
text-gray-400">
                                            Avg: {(a.metrics.total_duration /
a.metrics.calls).toFixed(0)}ms
                                        </span>
                                    </>
                                )}
                            </div>
                        </div>
                        <span className="text-xs px-3 py-1 rounded-full bg-green-900/30
text-green-400 border border-green-700/30">
                            ● Active
                        </span>
                    </div>
                ))}

                <button
                    onClick={designAgent}
                    className="w-full py-3 rounded-xl border border-dashed
border-purple-500/30 text-purple-400 hover:bg-purple-500/10 transition-all"
                >
                    + Design New Agent
                </button>
            </div>
        )}

        {tab === "quantum" && (
            <div className="space-y-4">
                <div className="text-xs font-bold text-purple-300 mb-4">QUANTUM
REALITY ENGINE</div>
```

```
                    {quantumRealities.length === 0 && (
                      <div className="text-gray-500 text-center py-8">
                        No predictions active. Ask Jasper to predict or simulate scenarios.
                      </div>
                    )}

                    {quantumRealities.map((r, i) => (
                      <div key={i} className="quantum-reality rounded-xl p-4">
                        <div className="flex justify-between items-start mb-2">
                          <span className="font-bold text-emerald-200">{r.title}</span>
                          <div className="flex items-center gap-2">
                            <div className="w-16 h-2 bg-gray-800 rounded-full
overflow-hidden">
                              <div
                                className="h-full bg-emerald-500 transition-all"
                                style={{width: `${r.adjusted_probability || r.probability}%`}}
                              ></div>
                            </div>
                            <span className="text-xs text-emerald-400 font-mono">
                              {r.adjusted_probability || r.probability}%
                            </span>
                          </div>
                        </div>
                        <div className="text-xs text-gray-400 mb-2">{r.description}</div>
                        {r.key_assumptions && (
                          <div className="flex flex-wrap gap-1">
                            {r.key_assumptions.map((ass, j) => (
                              <span key={j} className="text-xs px-2 py-0.5 rounded
bg-emerald-900/20 text-emerald-400/70 border border-emerald-700/20">
                                {ass}
                              </span>
                            ))}
                          </div>
                        )}
                      </div>
                    ))}
                  </div>
                )}

                {tab === "evolution" && (
                  <div className="space-y-4">
                    <div className="flex justify-between items-center mb-4">
```

```
                        <span className="text-xs font-bold text-purple-300">PENDING
ARCHITECTURAL CHANGES</span>
                        <span className="text-xs text-gray-500">Cycle:
{evolution.cycle_count}</span>
                    </div>

                    {pendingEvolutions.length === 0 && (
                        <div className="text-gray-500 text-center py-8 border border-dashed
border-gray-800 rounded-xl">
                            No pending evolutions. System is stable.
                        </div>
                    )}

                    {pendingEvolutions.map((imp, idx) => (
                        <div key={idx} className="evolution-card rounded-xl p-4 border-l-4
border-l-purple-500">
                            <div className="flex justify-between items-start mb-2">
                                <div>
                                    <div className="font-bold text-purple-200
text-sm">{imp.action.toUpperCase()}: {imp.target}</div>
                                    <div className="text-xs text-gray-400
mt-1">{imp.rationale}</div>
                                </div>
                                <button
                                    onClick={() => applyEvolution(idx)}
                                    className="px-3 py-1 rounded bg-green-600/20 text-green-400
text-xs border border-green-500/30 hover:bg-green-600/30 transition-all"
                                >
                                    Apply
                                </button>
                            </div>
                            <div className="text-xs text-gray-500 font-mono mt-2 p-2 bg-black/30
rounded">
                                {JSON.stringify(imp.parameters, null, 2).substring(0, 200)}...
                            </div>
                        </div>
                    ))}
                </div>
            )}

            {tab === "mother" && (
                <div className="space-y-4">
                    <div className="text-xs font-bold text-red-300 mb-4 flex items-center
gap-2">
```

```
                        <span className="w-2 h-2 rounded-full bg-red-500
animate-pulse"></span>
                        MOTHER NANUET OVERSIGHT
                    </div>

                    {motherStatus ? (
                        <>
                            <div className="grid grid-cols-2 gap-4 mb-4">
                                <div className="bg-black/30 p-3 rounded-lg border
border-red-500/20">
                                    <div className="text-2xl font-bold
text-red-400">{motherStatus.violations_24h}</div>
                                    <div className="text-xs text-gray-500">Violations (24h)</div>
                                </div>
                                <div className="bg-black/30 p-3 rounded-lg border
border-purple-500/20">
                                    <div className="text-2xl font-bold
text-purple-400">{motherStatus.total_alerts}</div>
                                    <div className="text-xs text-gray-500">Total Alerts</div>
                                </div>
                            </div>

                            <div className="space-y-2">
                                <div className="text-xs text-gray-500 uppercase
tracking-wider">Recent Alerts</div>
                                {motherStatus.recent_alerts?.length === 0 && (
                                    <div className="text-gray-600 text-sm">No recent alerts. All
systems nominal.</div>
                                )}
                                {motherStatus.recent_alerts?.map((alert, i) => (
                                    <div key={i} className="mother-alert p-3 rounded text-xs">
                                        <div className="text-red-300 font-bold">{new
Date(alert.time).toLocaleTimeString()}</div>
                                        <div className="text-gray-300 mt-1">{alert.message}</div>
                                    </div>
                                ))}
                            </div>

                            <div className="mt-4 p-3 rounded-lg bg-green-900/10 border
border-green-700/20">
                                <div className="text-xs text-green-400 font-bold mb-1">CORE
DIRECTIVES</div>
                                <div className="text-xs
text-gray-400">{motherStatus.core_directives_status}</div>
```

```
                            </div>
                        </>
                    ) : (
                        <div className="text-gray-500">Loading Mother status...</div>
                    )}
                </div>
            )}
        </div>
    </div>
  );
}

ReactDOM.createRoot(document.getElementById('root')).render(<JasperNaNuet />);
</script>
</body>
</html>
```

3. Docker Configuration

`Dockerfile`:
```dockerfile
FROM python:3.11-slim

WORKDIR /app

# Install system dependencies
RUN apt-get update && apt-get install -y \
    ffmpeg \
    libsndfile1 \
    gcc \
    g++ \
    && rm -rf /var/lib/apt/lists/*

# Copy requirements
COPY requirements.txt .

# Install Python dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy application
COPY jasper_backend_v7.py .
COPY chroma_db/ ./chroma_db/ 2>/dev/null || true
```

```
# Create volume for persistent data
VOLUME ["/app/data"]

# Expose port
EXPOSE 8080

# Health check
HEALTHCHECK --interval=30s --timeout=10s --start-period=40s --retries=3 \
    CMD python -c "import requests; requests.get('http://localhost:8080/health')" || exit 1

# Run with multiple workers (for production, use gunicorn)
CMD ["uvicorn", "jasper_backend_v7:app", "--host", "0.0.0.0", "--port", "8080", "--workers", "1"]
```

`requirements.txt`:
```
fastapi==0.104.1
uvicorn[standard]==0.24.0
python-jose[cryptography]==3.3.0
python-multipart==0.0.6
slowapi==0.1.9
chromadb==0.4.18
sentence-transformers==2.2.2
torch==2.1.0
speechbrain==0.5.16
openai-whisper==20231117
soundfile==0.12.1
numpy==1.24.3
google-auth==2.23.4
google-auth-oauthlib==1.1.0
google-auth-httplib2==0.1.1
google-api-python-client==2.108.0
httpx==0.25.2
pydantic==2.5.0
```

`docker-compose.yml`:
```yaml
version: '3.8'

services:
  jasper:
    build: .
    ports:
```

```
      - "8080:8080"
    environment:
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
      - GOOGLE_DRIVE_CREDENTIALS=${GOOGLE_DRIVE_CREDENTIALS}
      - JWT_SECRET_KEY=${JWT_SECRET_KEY}
      - JASPER_API_KEY=${JASPER_API_KEY}
    volumes:
      - jasper_data:/app/data
      - ./jasper_memory.db:/app/jasper_memory.db
      - ./chroma_db:/app/chroma_db
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
      interval: 30s
      timeout: 10s
      retries: 3

volumes:
  jasper_data:
```

--